

Maven Step-by-Step Guide

A beginner-friendly PDF handout for understanding and using Apache Maven

What is Maven? Apache Maven is a build automation and dependency management tool mainly used for Java projects. It helps you compile code, manage libraries, run tests, package applications, and organize projects using a standard structure and a central configuration file named pom.xml.

1. Install Java first

- Before using Maven, make sure Java is installed on your system.
- Open a terminal or command prompt and type: `java -version`
- If Java is installed correctly, you will see the installed JDK version.

2. Install Maven

- Download Maven from the official Apache Maven website.
- Extract the downloaded folder to a suitable location on your computer.
- Add Maven's bin folder to the system PATH variable.
- Check the installation by typing: `mvn -version`
- If installed correctly, Maven will display its version along with Java details.

3. Understand the project structure

- Maven follows a standard directory layout so that all developers can work in the same organized format.

Common structure:

- `src/main/java` -> application source code
- `src/test/java` -> test source code
- `src/main/resources` -> configuration files and resources
- `pom.xml` -> project configuration file

4. Create a Maven project

- You can create a simple Maven project from the terminal using an archetype command.

Command:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=myapp  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

- This creates a starter Java project with the standard Maven layout.

5. Learn the role of pom.xml

- The pom.xml file is the heart of a Maven project.
- It stores the project name, version, dependencies, plugins, and build instructions.

Important parts inside pom.xml usually include:

- groupId: identifies the organization or package group
- artifactId: identifies the project name
- version: current version of the project
- dependencies: external libraries needed by the project

6. Add dependencies

- Instead of downloading JAR files manually, Maven downloads required libraries automatically from repositories such as Maven Central.

Example dependency for JUnit testing:

<dependency>

- <groupId>junit</groupId>
- <artifactId>junit</artifactId>
- <version>4.13.2</version>
- <scope>test</scope>

</dependency>

7. Run common Maven commands

Use these commands inside the project folder:

- mvn compile -> compiles the source code
- mvn test -> runs unit tests
- mvn package -> creates a JAR or WAR file
- mvn install -> installs the package into the local repository
- mvn clean -> removes old build files from the target folder

8. Understand the build lifecycle

- Maven works through phases called the build lifecycle.

The most common lifecycle order is:

validate -> compile -> test -> package -> verify -> install -> deploy

- When you run a later phase such as mvn package, Maven automatically runs the earlier required phases first.

9. Build your first project

- Navigate to your Maven project folder.
- Type: `mvn package`
- Maven will download any missing dependencies, compile the project, run tests, and create an output file inside the target folder.

10. Why Maven is useful

- Maven saves time by automating repetitive tasks.
- It ensures every team member follows the same project structure.
- It makes dependency handling simple and reliable.
- It is widely used in enterprise Java and Spring Boot development.

11. Quick sample pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
```

- `<modelVersion>4.0.0</modelVersion>`
- `<groupId>com.example</groupId>`
- `<artifactId>myapp</artifactId>`
- `<version>1.0-SNAPSHOT</version>`
- `<dependencies>`
- `<dependency>`
- `<groupId>org.springframework</groupId>`
- `<artifactId>spring-context</artifactId>`
- `<version>5.3.30</version>`
- `</dependency>`
- `</dependencies>`

```
</project>
```

12. Final summary

- Maven is a tool that manages project building, dependencies, testing, and packaging through a single pom.xml file. Once you learn the standard structure and a few commands, it becomes much easier to create and maintain Java applications in a professional way.

Beginner checklist

- Install Java
- Install Maven and verify with `mvn -version`
- Create a project using Maven archetype
- Understand pom.xml
- Add dependencies
- Run `mvn compile`, `mvn test`, and `mvn package`

Prepared as a simple study guide